Ayudantía 7 · Big Data · Recuperación de Información

SIMILITUD Y OPTIMIZACIÓN

Roberto Ibaceta Cisternas

ÍNDICE

01. Dónde quedamos

02. Jaccard, distancia y coseno

03. Comparar la consulta con el corpus

04. Agrupación espacial

05. Funciones hash

06. Errores típicos y cierre

Dónde quedamos

Vector TF-IDF	consult	dat	disc	hadoop	indic	memori	proces	rap	spark
Q	0,3	0	0	0	0	0,3	0	0,6	0

Q = «consultas rápidas en memoria», traducida igual que los documentos: limpiar, contar y multiplicar por el IDF.

Como conjuntos

$Q = \{consult, memori, rap\}$
$C_1 = \{dat, memori, proces, spark\}$
$C_2 = \{dat, disc, hadoop, proces\}$
$C_3 = \{consult, dat, indic\}$
$C_4 = \{consult, dat, indic, memori, rap\}$

Medida de similitud

¿Cómo comparamos?

- Intersección de conjuntos
- Distancia espacial
- Alineación direccional
- Probabilidades

Optimización

¿Cómo hacerlo eficiente?

- Agrupación espacial
- Funciones hash

Representación: lista. Hoy, la pregunta es otra: ¿qué documento responde mejor a Q?

Intersección de conjuntos: Jaccard

$$J(A, B) = |A \cap B| / |A \cup B|$$

El ejemplo del profesor

C_1 = {vec, disp}
C_2 = {vec, densos}
$C_1 \cap C_2$ = {vec}
$C_1 \cup C_2$ = {vec, disp, densos}
J = 1/3 ≈ 0,33

Va de **0** (nada en común) a **1** (idénticos). Más alto = más parecido.

Con Q = {consult, memori, rap}

Doc	Q ∩ C	|Q ∪ C|	J(Q, C)
d_1	{memori}	6	1/6 ≈ 0,17
d_2	∅ (nada)	7	0
d_3	{consult}	5	1/5 = 0,2
d_4	{consult, memori, rap}	5	3/5 = 0,6 · 1.º

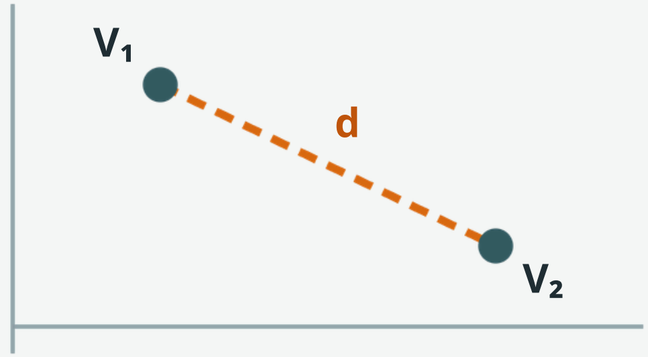
Ojo: Jaccard no ve pesos. Para él «dat» vale lo mismo que «rap», y lo que ganamos con el IDF se pierde.

Distancia espacial: euclidiana

$$d(x, y) = \sqrt{\left(\Sigma (x_i - y_i)^2 \right)}$$

Ejemploo

$V_1 = (1, 1, 0)$
$V_2 = (1, 0, 1)$
restas = (0, 1, -1)
cuadrados = (0, 1, 1) → suma 2
d = √2 ≈ 1,41



Va de **0** (idénticos) hacia arriba. **Más chico = más parecido**, al revés que Jaccard.

Truco: para ordenar no necesitas la raíz. Compara las distancias al cuadrado.

Alineación direccional: coseno

$$\cos(x, y) = (x \cdot y) / (\|x\| \cdot \|y\|)$$

$$x \cdot y = \sum x_i \, y_i$$

$$\|x\| = \sqrt{\left(\sum x_i^2 \right)}$$

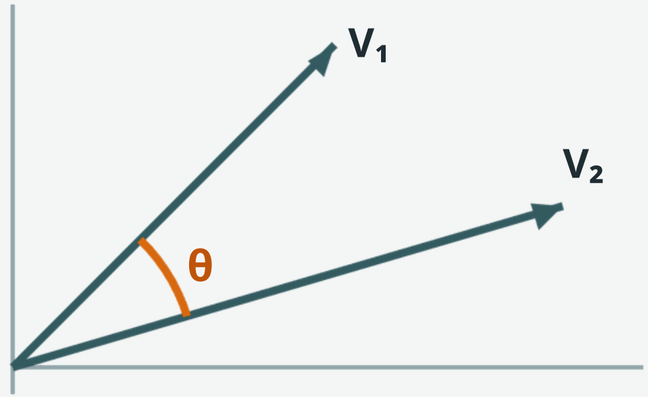
Ejemplo

$V_1 \cdot V_2 = 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 = 1$

$\|V_1\| = \|V_2\| = \sqrt{2}$

$\cos = 1 / (\sqrt{2} \cdot \sqrt{2}) = 1/2 = 0{,}5$

No mide qué tan lejos están, sino **hacia dónde apuntan**. Con pesos TF-IDF va de **0** (nada en común) a **1** (misma dirección). **Más alto = más parecido.**



Actividad 1: ¿qué documento se parece más a Q? 8 min

Truco sin calculadora: todos los pesos son 0, 0,3 o 0,6. Dividimos por 0,3 y quedan enteros. El coseno no cambia y el orden por distancia tampoco.

En unidades de 0,3	consult	dat	disc	hadoop	indic	memori	proces	rap	spark
Q	1	0	0	0	0	1	0	2	0
d_1	0	0	0	0	0	1	1	0	2
d_2	0	0	2	2	0	0	1	0	0
d_3	1	0	0	0	1	0	0	0	0
d_4	2	0	0	0	1	1	0	2	0

Para cada documento, calcula

1) el producto punto Q · d · 2) la distancia² · 3) el coseno · 4) el ranking

Datos

$||Q||^2 = 6 \cdot \sqrt{12} \approx 3{,}5 \cdot \sqrt{60} \approx 7{,}7$

Solución 1: d_4 gana con las tres medidas

Lugar	Doc	Q · d	Distancia²	Coseno	Jaccard
1.º	d_4	7	2	7 / √60 ≈ 0,9	0,6
2.º	d_3	1	6	1 / √12 ≈ 0,29	0,2
3.º	d_1	1	10	1 / √36 = 1/6 ≈ 0,17	0,17
4.º	d_2	0	15	0	0

Por qué gana d_4

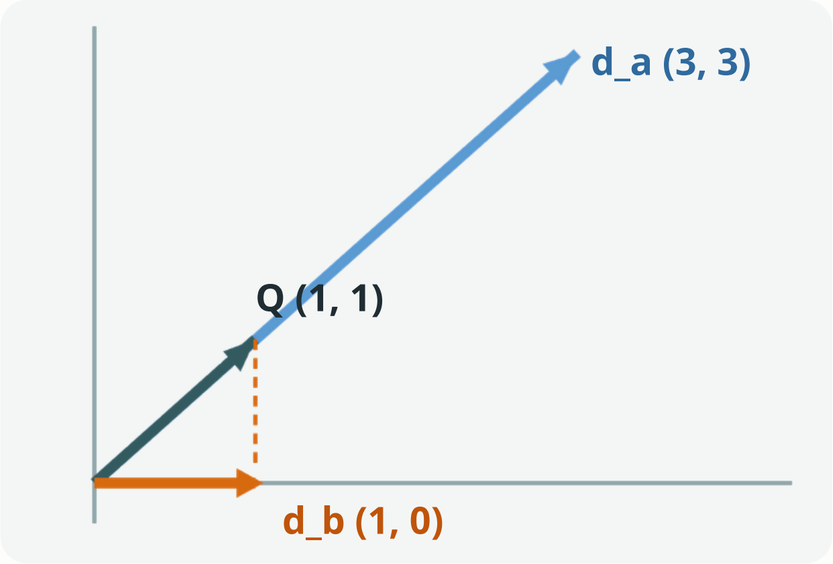
Comparte «consult», «memori» y «rap» con la consulta, y «rap» pesa doble.

Mira d_1 y d_3

Tienen el mismo producto punto (1), pero d_3 queda arriba porque es más corto. Eso hace la división por las normas.

¿Distancia o ángulo?

Dos términos: spark (horizontal) y hadoop (vertical)



d_a es un documento largo que habla de lo mismo que Q, tres veces más. **d_b** es corto y solo habla de spark.

Doc	Distancia a Q	Coseno	Gana según
d_a (3, 3)	√8 ≈ 2,83	1	Coseno
d_b (1, 0)	1	1/√2 ≈ 0,71	Distancia

Por eso en texto se usa el coseno

La distancia castiga a d_a solo por ser largo. El coseno mira la dirección y ve que trata exactamente lo mismo que Q: no lo afecta el largo del documento.

Resumen: cuatro formas de comparar

Medida	Usa	Qué mide	Rango	Más parecido
Jaccard	Conjuntos	Términos compartidos	0 a 1	Más alto
Euclidiana	Vectores	Distancia entre puntos	0 en adelante	Más bajo
Coseno	Vectores	Ángulo entre vectores	0 a 1	Más alto
Probabilidades	Modelo probabilístico	Probabilidad de relevancia	0 a 1	Más alto

Probabilidades: solo la idea

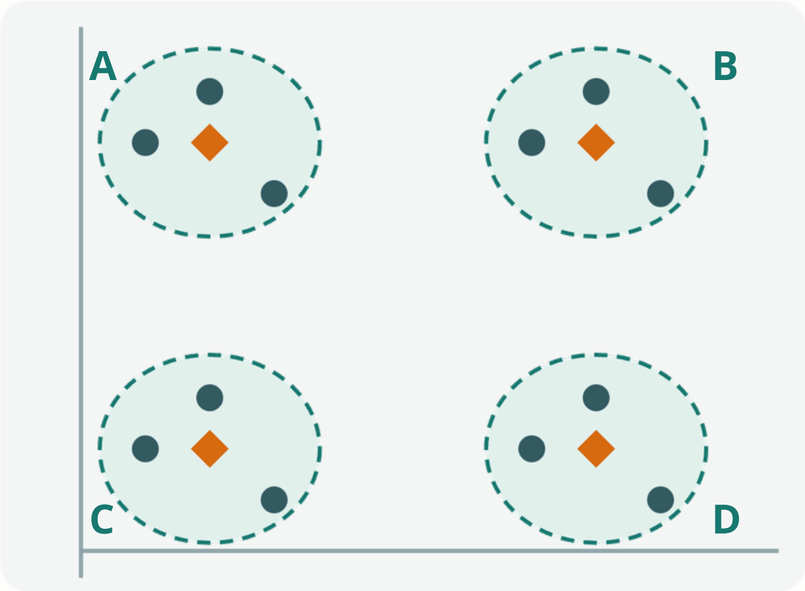
Cambia la pregunta: no «¿qué tan parecidos son?», sino «¿qué tan probable es que d sea relevante para Q?». Los buscadores usan variantes como BM25, pariente del TF-IDF.

Error más común

Ordenar la distancia al revés: ahí gana el número más chico.

Optimización 1: agrupación espacial

12 elementos, 4 grupos (como en la pizarra)



Puntos: elementos. Rombos naranjos: centroides de cada grupo.

1.000.000 documentos = 1.000.000 de comparaciones por cada consulta. Optimizar es comparar con menos.

k-means en tres pasos

1. Elegir k centros iniciales.
2. Asignar cada punto al centro más cercano.
3. Recalcular cada centro como el promedio de su grupo, y repetir hasta que no cambie.

Para buscar: comparo la consulta con los centros y después solo con los puntos del grupo ganador.

Actividad 2: buscar con grupos

5 min

Encuentra el punto más cercano a **Q = (7, 6)** usando los grupos, y cuenta cuántas comparaciones hiciste.

Grupo	Puntos	Centroide
A	(1, 8) · (2, 9) · (3, 7)	(2, 8)
B	(7, 8) · (8, 9) · (9, 7)	(8, 8)
C	(1, 2) · (2, 3) · (3, 1)	(2, 2)
D	(7, 2) · (8, 3) · (9, 1)	(8, 2)

Pasos

1. Distancia² de Q a los 4 centroides.
2. Distancia² de Q a los puntos del grupo ganador.
3. Cuenta las comparaciones.

$$d^2 = (\Delta x)^2 + (\Delta y)^2$$

sin raíces

Solución 2: menos comparaciones

Paso 1 · contra los 4 centroides

`A: 25 + 4 = 29 · B: 1 + 4 = 5`

`C: 25 + 16 = 41 · D: 1 + 16 = 17`

Paso 2 · contra los 3 puntos de B

`(7, 8): 4 · (8, 9): 10 · (9, 7): 5`

7 comparaciones en vez de 12, y el vecino es el mismo: **(7, 8)**.

A escala

1.000.000 de documentos en 1.000 grupos de 1.000: unas **2.000** comparaciones por consulta en vez de 1.000.000.

La letra chica: Q = (5, 8)

Empata entre A y B (9 y 9). Si eliges A, encuentras (3, 7) con 5, pero el más cercano real es (7, 8) con 4. La búsqueda por grupos es **aproximada**: en la práctica se revisan los 2 o 3 grupos más cercanos.

Optimización 2: funciones hash

Una función hash normal manda cosas distintas a lugares distintos. Aquí queremos lo contrario: que **lo parecido caiga en el mismo balde**, y comparar solo dentro de cada balde.

MinHash en una línea

Elige un orden al azar del vocabulario. La **firma** de un conjunto es su primer término en ese orden.

Ejemplo, orden h_1: spark, consult, rap, hadoop, dat, memori, disc, indic, proces

C_3 = {consult, dat, indic}
spark no está · consult sí → **firma = consult**

Los cuatro documentos con h_1



Lo clave: la probabilidad de que dos conjuntos tengan la misma firma es su Jaccard.

Actividad 3: firmas MinHash

5 min

```
C₃ = {consult, dat, indic}
C₄ = {consult, dat, indic, memori, rap}
```

Con cada orden, escribe la firma de C_3 y de C_4 y marca si coinciden. **Antes de empezar:** ¿en cuántas de las 5 crees que van a coincidir?

Orden	Vocabulario en ese orden	Firma C_3	Firma C_4	¿Iguales?
h_1	spark, consult, rap, hadoop, dat, memori, disc, indic, proces			
h_2	hadoop, rap, dat, spark, consult, indic, proces, memori, disc			
h_3	proces, disc, dat, memori, spark, rap, consult, hadoop, indic			
h_4	memori, consult, spark, indic, disc, dat, rap, proces, hadoop			
h_5	spark, hadoop, indic, rap, proces, consult, memori, dat, disc			

Recuerda: la firma es el primer término del orden que **sí** está en el conjunto. Dato de la clase pasada: $J(C_3, C_4) = 0{,}6$.

Solución 3: la firma estima Jaccard

Orden	Firma C_3	Firma C_4	¿Iguales?
h_1	consult	consult	Sí
h_2	dat	rap	No
h_3	dat	dat	Sí
h_4	consult	memori	No
h_5	indic	indic	Sí

3 / 5

= 0,6 = **$J(C_3, C_4)$**. Coinciden justo cuando el primer término de la unión está en la intersección.

Es una estimación

Con estos mismos 5 órdenes, d_1 y d_2 coinciden 1 de 5 (0,2), pero su Jaccard real es 1/3. Con 100 órdenes la estimación se acerca mucho.

Para qué sirve: comparar firmas cortas es mucho más barato que comparar documentos enteros. Así se encuentran casi duplicados entre millones de páginas.

Para llevarse

4 errores típicos

1. **Ordenar la distancia al revés.** Gana la más chica.
2. **Usar solo el producto punto.** Sin dividir por las normas, ganan los documentos largos.
3. **Calcular Jaccard con pesos.** Jaccard usa conjuntos, no TF-IDF.
4. **Creer que agrupar o usar hash es exacto.** Es aproximado.

3 ideas

1. **Cada representación tiene su medida:** conjuntos con Jaccard; vectores con distancia o coseno.
2. **En texto se prefiere el coseno:** no lo afecta el **largo del documento**.
3. **Optimizar es comparar con menos,** a cambio de un resultado **aproximado**.

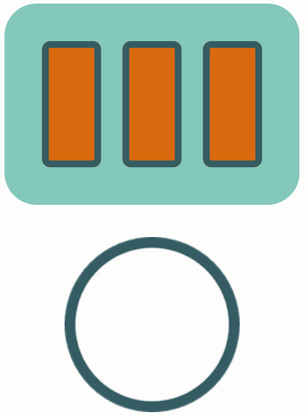
Con esto cerramos los tres pilares. Cátedra del 19 de octubre: recuperación de información moderna.

C O N T A C T O

Gmail
roberto.ibaceta@mail.udp.cl

Wsp
+569 42663436

Ds
roberibaz



MUCHAS GRACIAS

